Supervised Learning – Linear Regression

By Dr. Mehwish

Supervised Learning

- Supervised learning is a subcategory of machine learning.
- Supervised learning is the types of machine learning in which machines are trained using well "labeled" training data, and on the basis of that data, machines predict the output.
- As input data is fed into the model, it adjusts its weights until the model has been fitted appropriately, which occurs as part of the cross validation process.

Supervised Learning

- Supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm to learn the mapping function from the input to the output.

$$Y = f(X)$$

- The goal is to approximate the mapping function so well that when you have new input data (x) then you can predict the output variables (Y) for that data.

Supervised Learning

$$y = f(\mathbf{x})$$

output — prediction function — Image feature

- **Training:** given a *training set* of labeled examples $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, estimate the prediction function f by minimizing the prediction error on the training set
- **Testing:** apply f to a never before seen *test example* $\mathbf{x}$ and output the predicted value $y = f(\mathbf{x})$

Supervised Learning – Types

- Supervised learning can be separated into two types of problems when data mining—classification and regression:
- **Classification** uses an algorithm to accurately assign test data into specific categories.
- It recognizes specific entities within the dataset and attempts to draw some conclusions on how those entities should be labeled or defined.
- Common classification algorithms are linear classifiers, support vector machines (SVM), decision trees, k-nearest neighbor, and random forest algorithm.

Supervised Learning – Types

- Apply a prediction function to a feature representation of the image to get the desired output:

f( ) = “apple”

f( ) = “tomato”

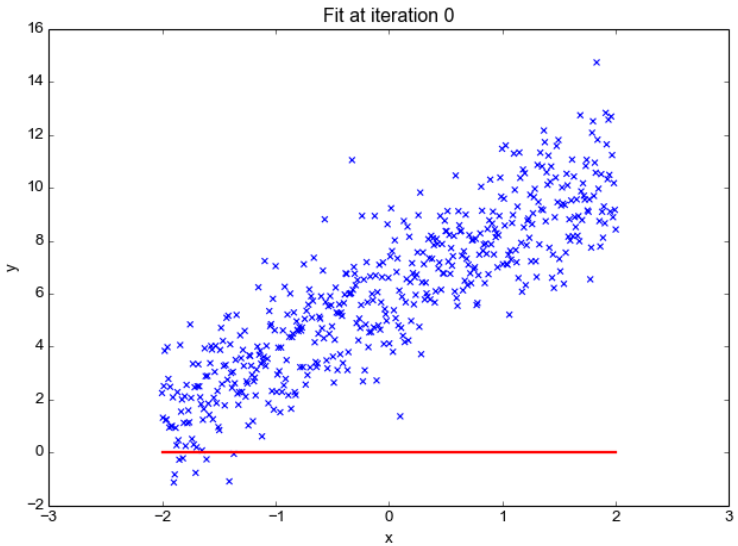
f( ) = “cow”

Supervised Learning – Types

- **Regression** is used to understand the relationship between dependent and independent variables.
- It is commonly used to make projections, such as for sales revenue for a given business.
- Linear regression, logistical regression, and polynomial regression

  are popular regression

  algorithms.

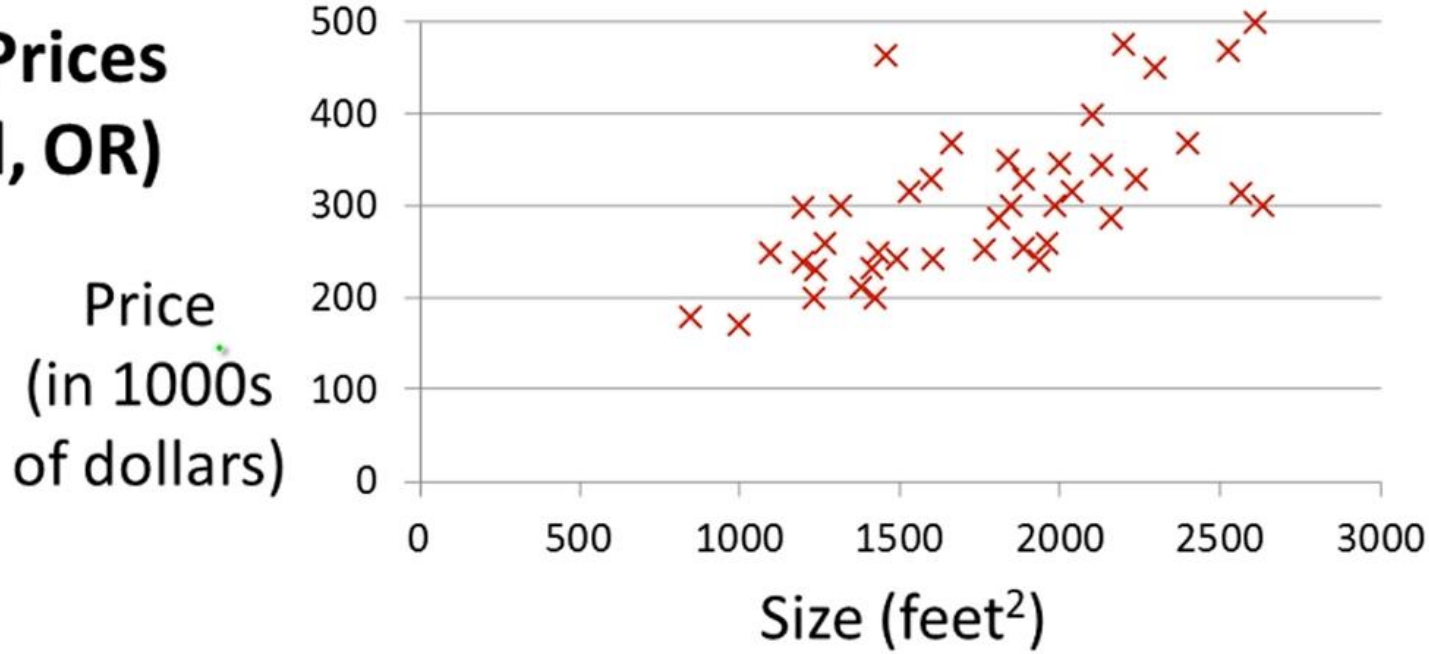


Supervised Learning – Linear Regression

- Linear regression analysis is used to predict the value of a variable based on the value of another variable.
- The variable you want to predict is called the dependent variable.
- The variable you are using to predict the other variable's value is called the independent variable.

Supervised Learning – Linear Regression

Housing Prices (Portland, OR)



Supervised Learning – Linear Regression

Size in feet^2 (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

Supervised Learning – Linear Regression

Notation:

m = Number of training examples

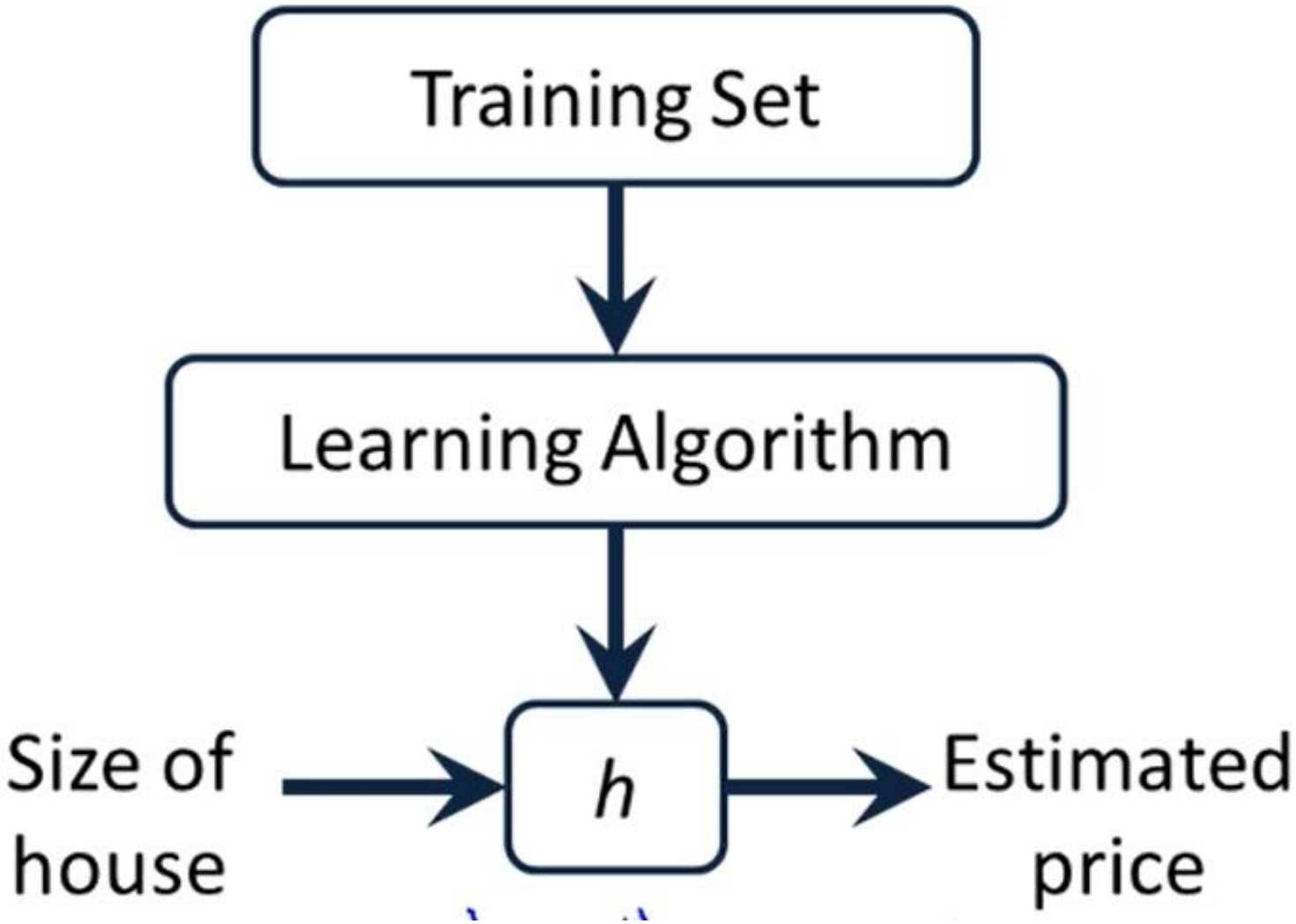
x's = "input" variable / features

y's = "output" variable / "target" variable

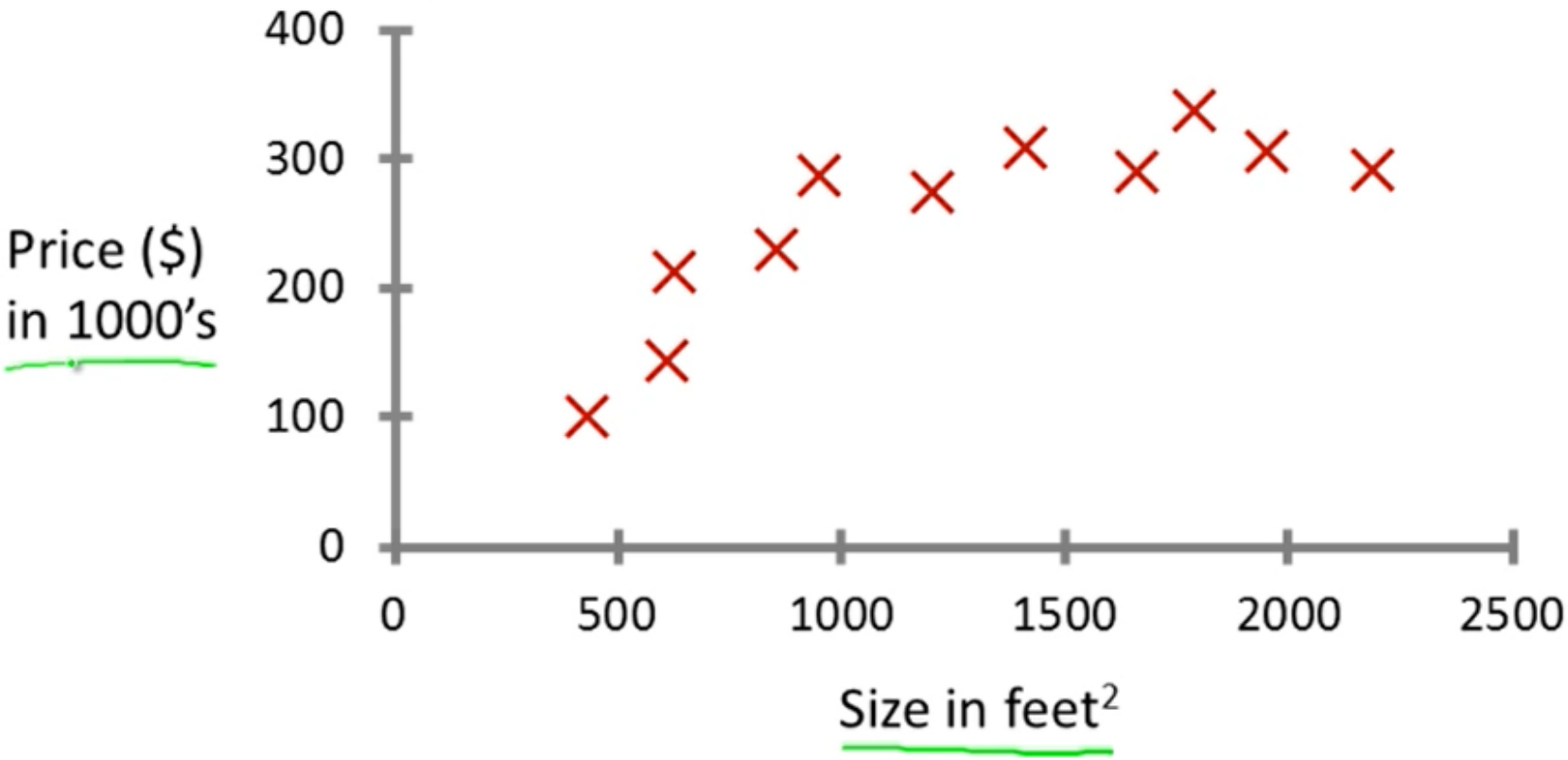
- (x,y) is one training example
- $(x^{(i)}, y^{(i)})$ is i^{th} Training Example
- $x^{(1)}=2104$ and $y^{(1)}=460$
- $x^{(2)}=1416$ and $y^{(2)}=232$

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

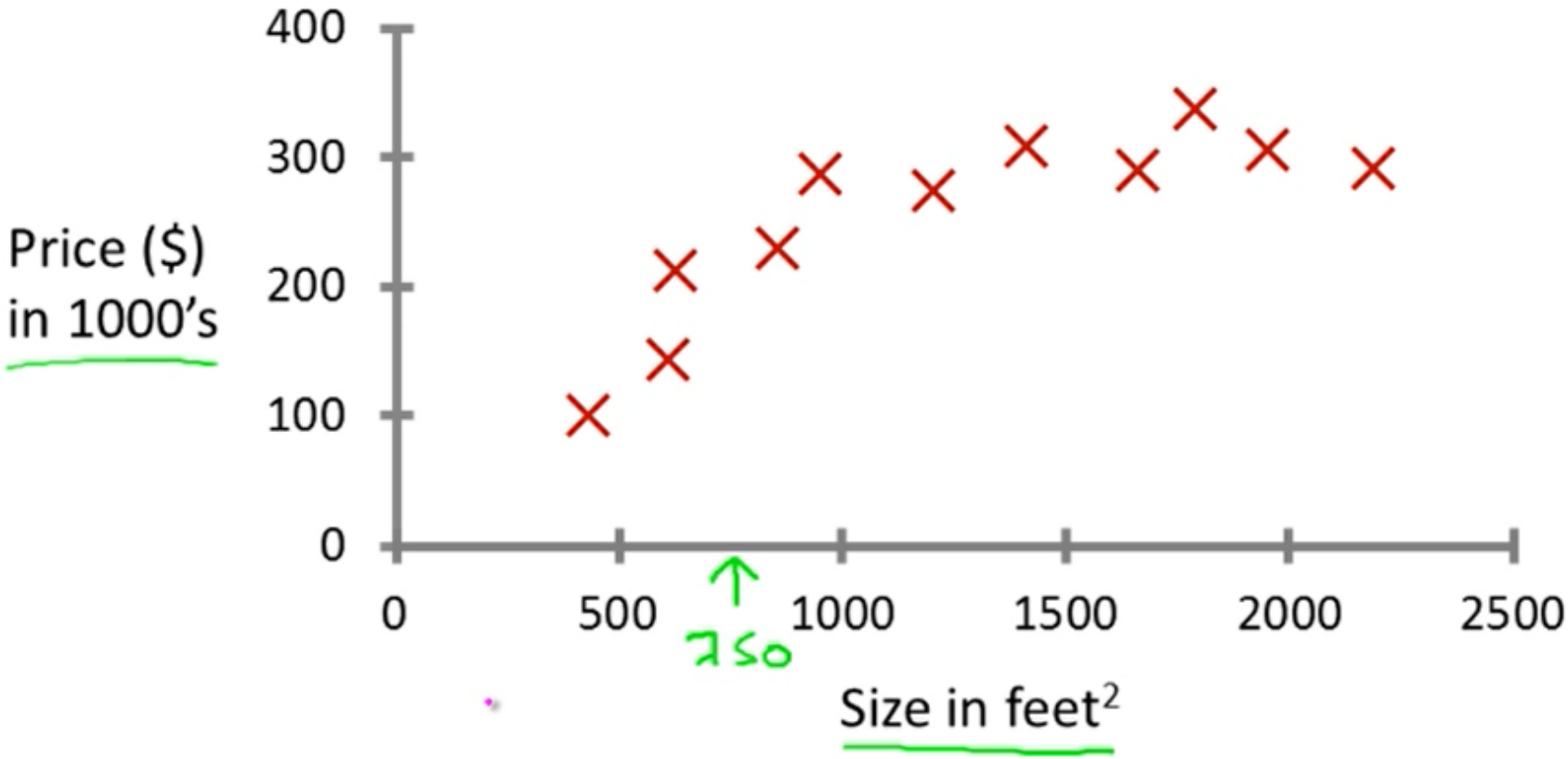
Supervised Learning – Linear Regression



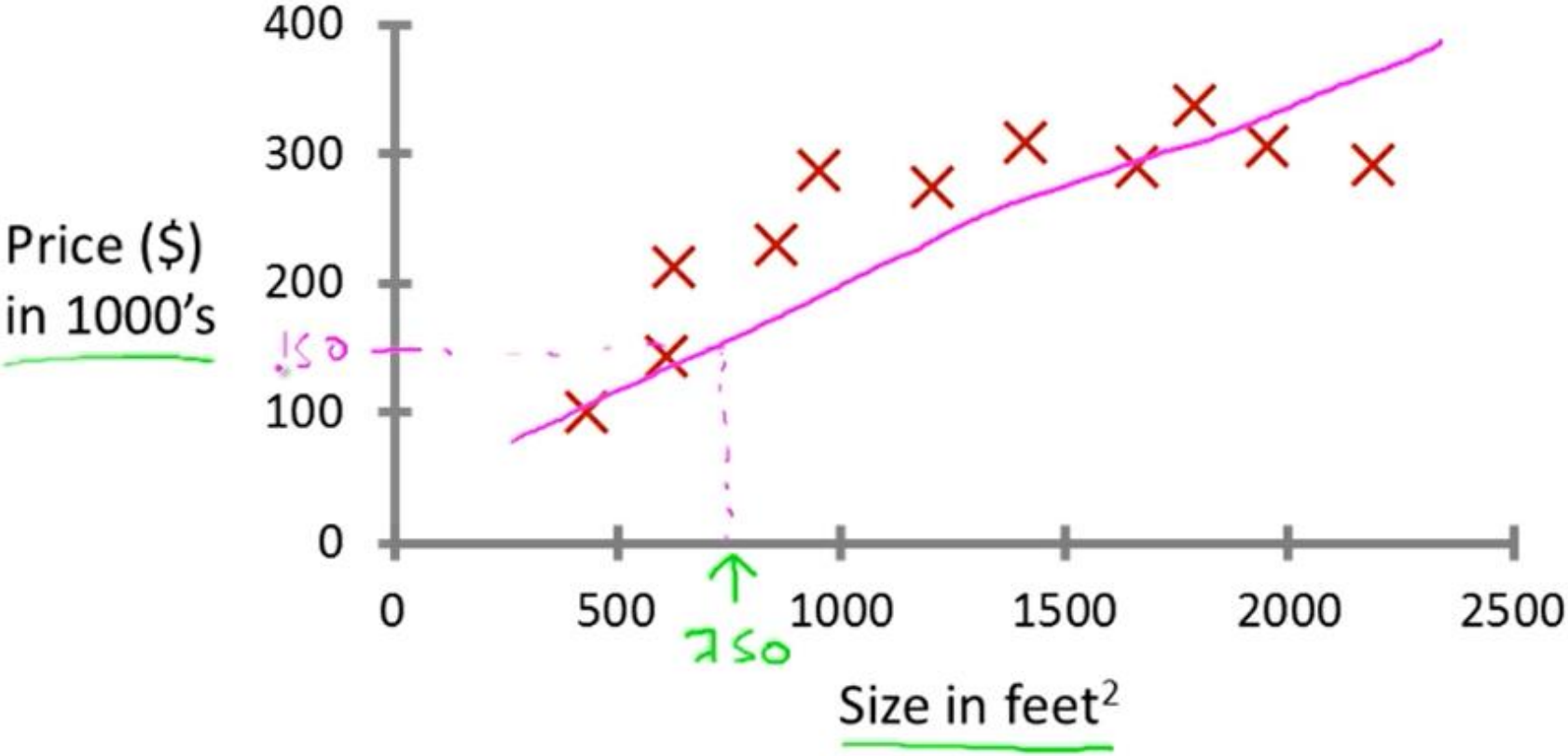
Supervised Learning – Linear Regression



Supervised Learning – Linear Regression

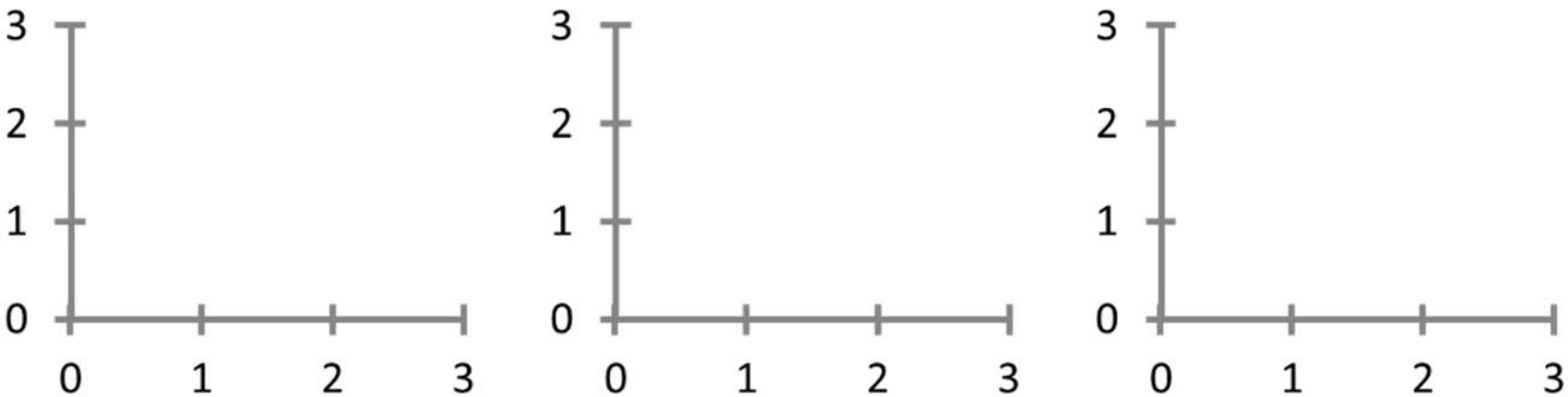


Supervised Learning – Linear Regression



Supervised Learning – Linear Regression

$$h_\theta(x) = \theta_0 + \theta_1 x$$



$\theta_0 = 1.5$
$\theta_1 = 0$

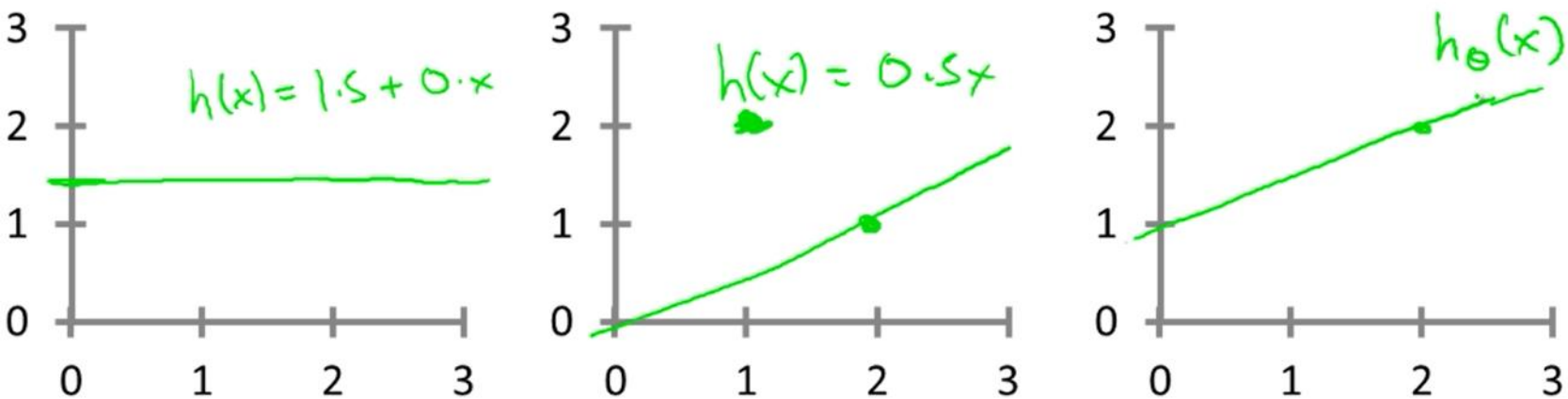
$\theta_0 = 0$
$\theta_1 = 0.5$

$\theta_0 = 1$
$\theta_1 = 0.5$

Activate Windows
Go to Settings to activate Windows.

Supervised Learning – Linear Regression

$$h_\theta(x) = \theta_0 + \theta_1 x$$



$\theta_0 = 1.5$
$\theta_1 = 0$

$\theta_0 = 0$
$\theta_1 = 0.5$

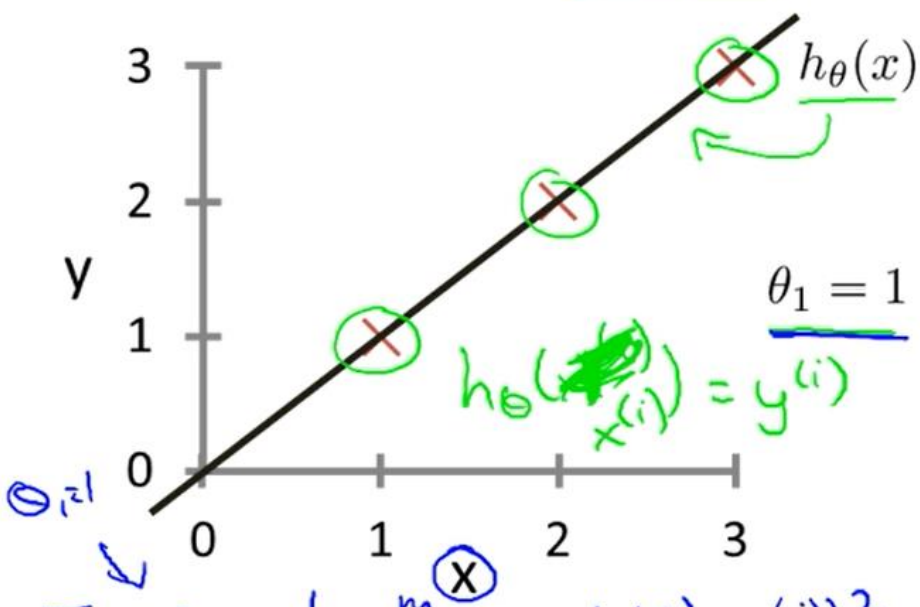
$\theta_0 = 1$
$\theta_1 = 0.5$

Activate Windows
Go to Settings to activate Windows.

Supervised Learning – Linear Regression (Cost Function)

$h_\theta(x)$

(for fixed θ_1, this is a function of x)

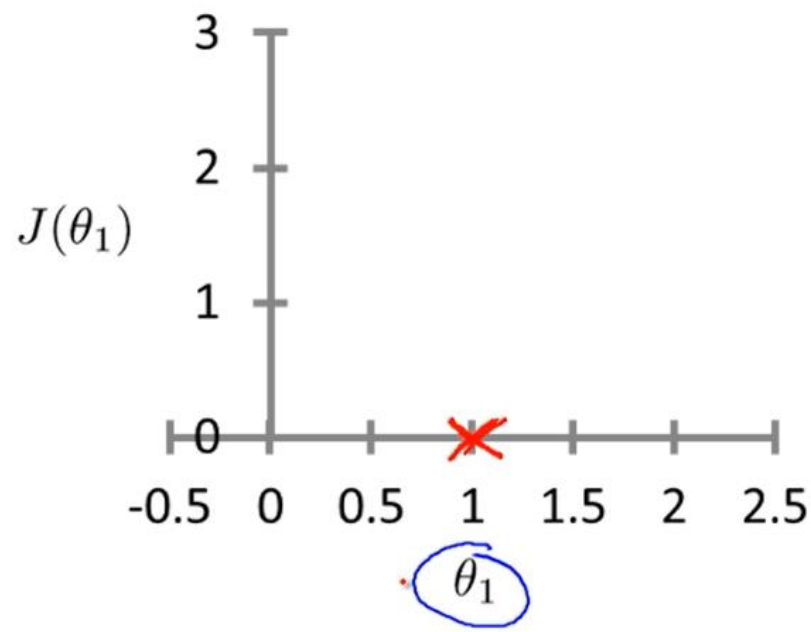


$h_\theta(x^{(i)}) = y^{(i)}$

$\theta_1 = 1$

$J(\theta_1)$

(function of the parameter θ_1)



$$J(\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

$$= \frac{1}{2m}\sum_{i=1}^{m}\left(\theta_1 x^{(i)} - y^{(i)}\right)^2 = \frac{1}{2m}\left(0^2 + 0^2 + 0^2\right) = 0^2$$

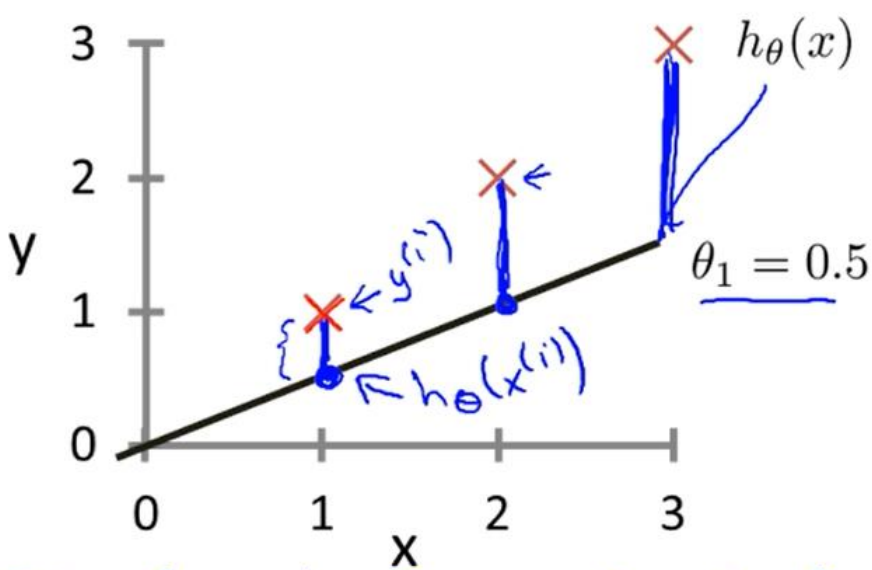
$J(1) = 0$

Activate Windows
Go to Settings to activate Windows.

Supervised Learning – Linear Regression (Cost Function)

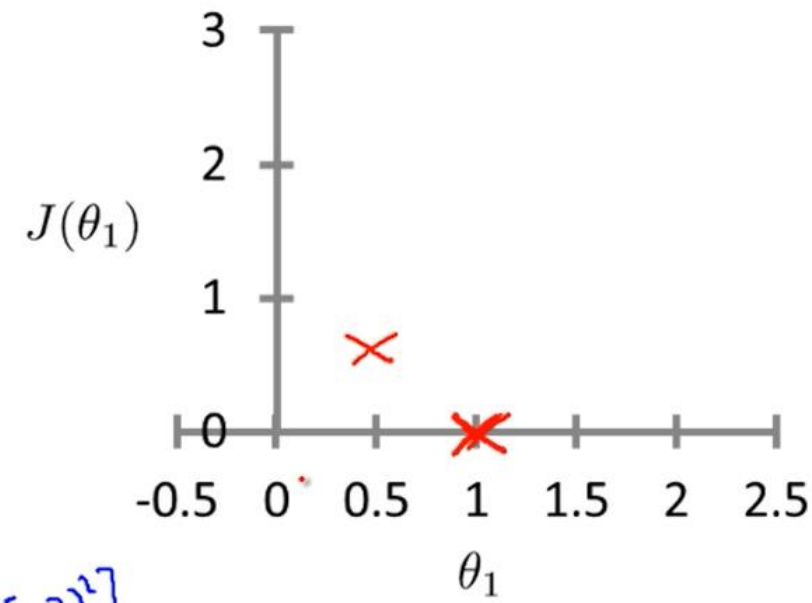
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

(function of the parameter θ_1)



$$J(0.5) = \frac{1}{2m}\left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right]$$

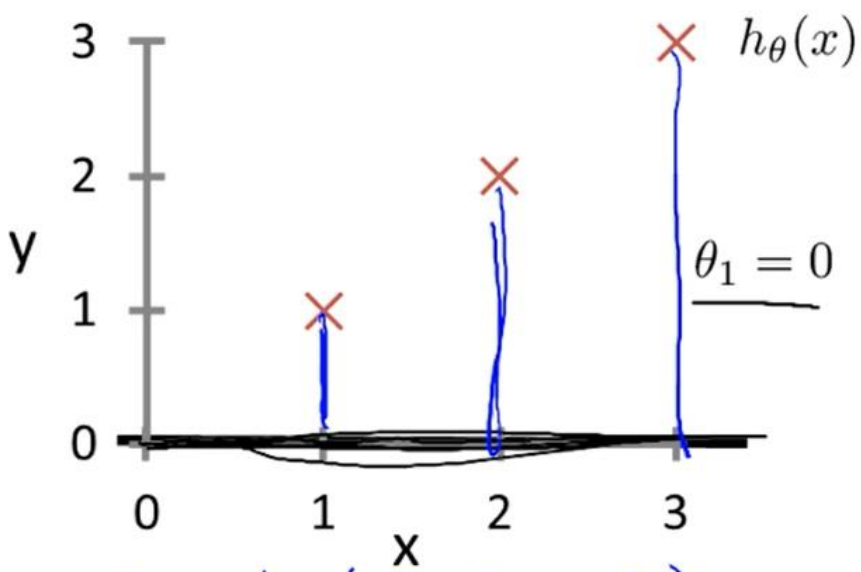
$$= \frac{1}{2\times 3}(3.5) = \frac{3.5}{6} \approx 0.58$$

Activate Windows
Go to Settings to activate Windows.

Supervised Learning – Linear Regression (Cost Function)

$h_\theta(x)$

(for fixed θ_1, this is a function of x)

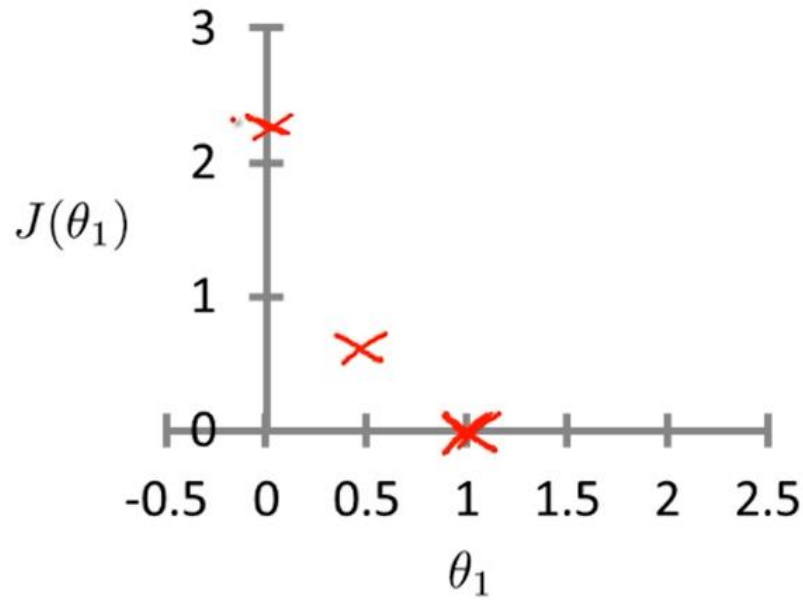


$J(0) = \frac{1}{2m}(1^2 + 2^2 + 3^2)$

$= \frac{1}{6} \cdot 14 \approx 2.3$

$J(\theta_1)$

(function of the parameter θ_1)

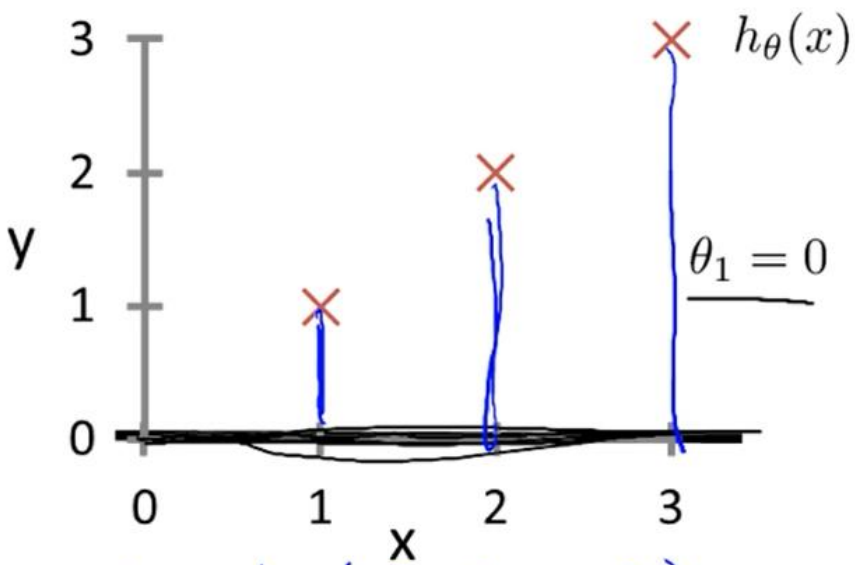


Activate Windows
Go to Settings to activate Windows.

Supervised Learning – Linear Regression (Cost Function)

$h_\theta(x)$

(for fixed θ_1, this is a function of x)

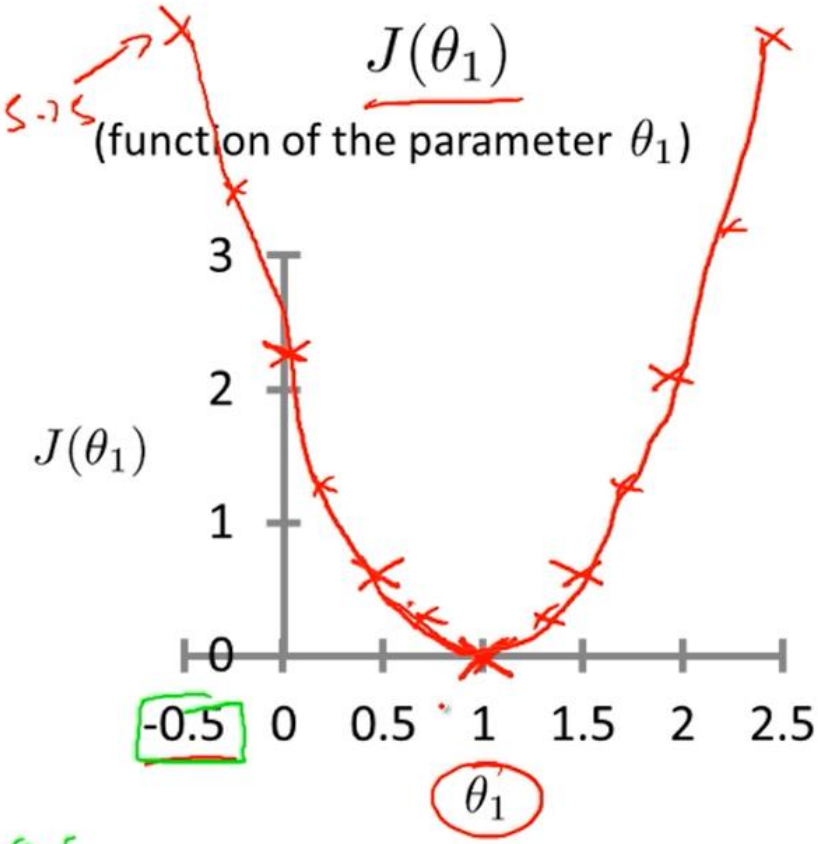


$J(0) = \frac{1}{2m}(1^2 + 2^2 + 3^2)$

$= \frac{1}{6} \cdot 14 \approx 2.3$

$J(\theta_1)$

(function of the parameter θ_1)



$h(x) = -0.5x$

Activate Windows
Go to Settings to activate Windows.

Linear Regression with Multiple Features

Size (feet^2)	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

Hypothesis:

Previously: $h_\theta(x) = \cancel{\theta_0 + \theta_1 x}$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

Feature Scaling

- Feature scaling is the process of normalizing the range of features in a dataset.
- Real-world datasets often contain features that are varying in degrees of magnitude, range, and units.
- Therefore, in order for machine learning models to interpret these features on the same scale, we need to perform feature scaling.

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40	nan	Yes
France	35	58000	Yes
Spain	nan	52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

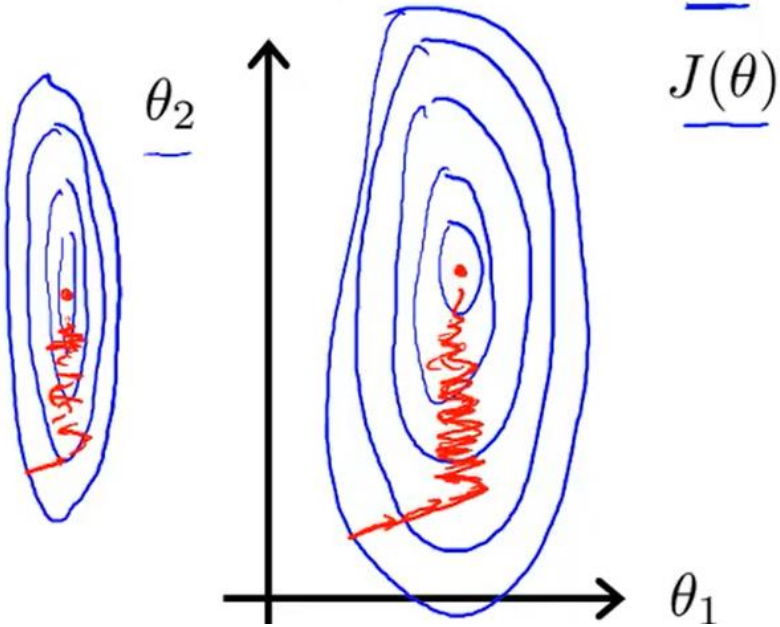
Feature Scaling

- If a feature in the dataset is **big in scale** (**Salary**) compared to **other (Age)** then in algorithms where Euclidean distance is measured this big scaled feature becomes **dominating** and **needs to be normalized**.
- To supress this effect, we need to bring all features to the **same level of magnitudes**.
- This can be achieved by **scaling**.
- Scaling the features will play a vital role in gradient descent algorithm.
- As **feature scaling** helps us to achieve the **gradient decent** more quickly.

Feature Scaling

E.g. x_1 = size (0-2000 feet^2)

x_2 = number of bedrooms (1-5)

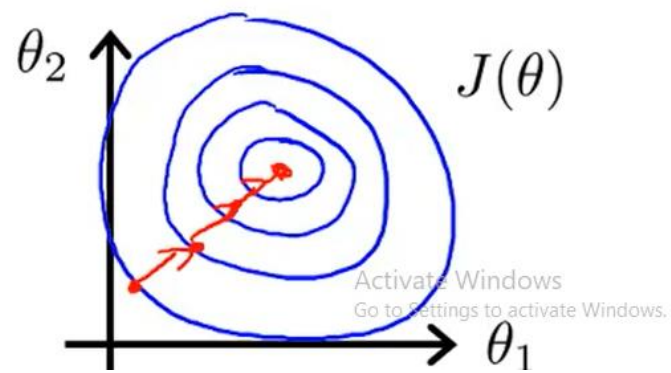


(a)

$$x_1 = \frac{\text{size (feet}^2)}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$

$0 \le x_1 \le 1 \qquad 0 \le x_2 \le 1$



(b)

Activate Windows
Go to Settings to activate Windows.

Feature Scaling

- Suppose you have only two parameters and one of the parameters can take a relatively **large range of values**.
- Then the **contour** of the cost function can look like very **tall** and **skinny ovals** (see blue ovals in the Figure (a)).
- Your gradients (the path of gradient is drawn in red) could take a long time and go back and forth to find the optimal solution.

- Instead if you scale your featured, the contour of the cost function might look like circles; then the gradient can take a much more straight path and achieve the optimal point much faster (see blue ovals in the Figure (b)).

Feature Scaling

Get every feature into approximately a $-1 \leq x_i \leq 1$ range.

$x_0 = 1$

$0 \leq x_1 \leq 3$ ✓

$-2 \leq x_2 \leq 0.5$ ✓

$-100 \leq x_3 \leq \boxed{100}$ ✗

$-0.0001 \leq x_4 \leq \boxed{0.0001}$ ✗

-3 to 3 ✓

$-\frac{1}{3}$ to $\frac{1}{3}$ ✓

Feature Scaling Types

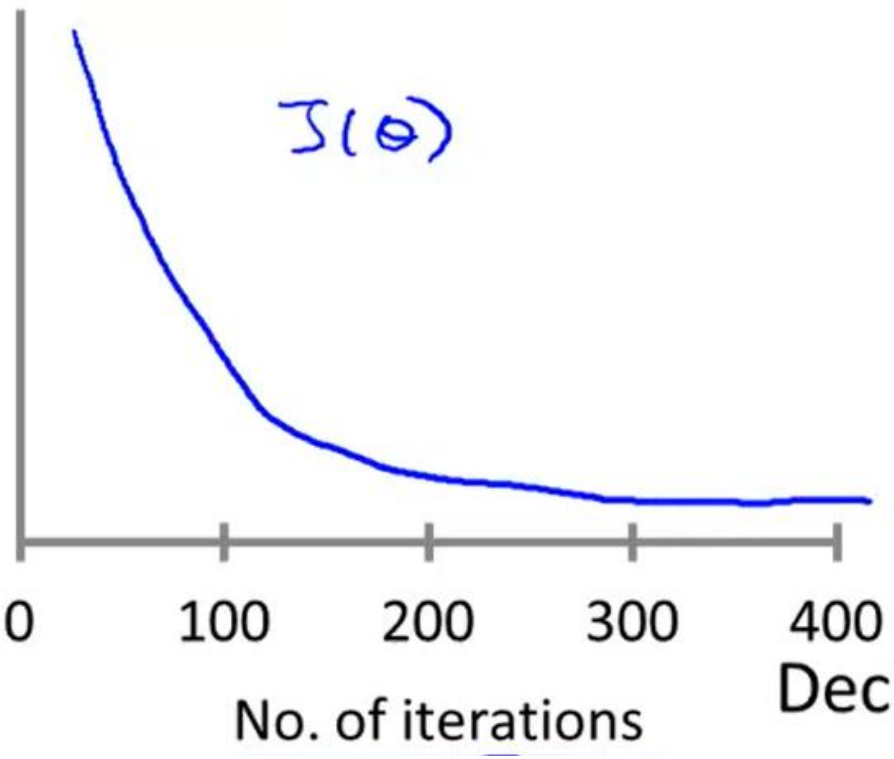
Standardization:

- Standardization replaces the values by their Z scores.
- Z score describes a value's relationship to the mean of a group of values.
- Z-scores may be positive or negative, with a positive value indicating the score is above the mean and a negative score indicating it is below the mean.
- If a z-score is 0, it indicates that the data point's score is identical to the mean score.

Practical Learning Rate

- The learning rate is a hyper-parameter that controls how much to change the model in response to the estimated error each time the model weights are updated
- Choosing the learning rate is challenging as a value too small may result in a long training process that could get stuck, whereas a value too large may result in learning a sub-optimal set of weights too fast or an unstable training process

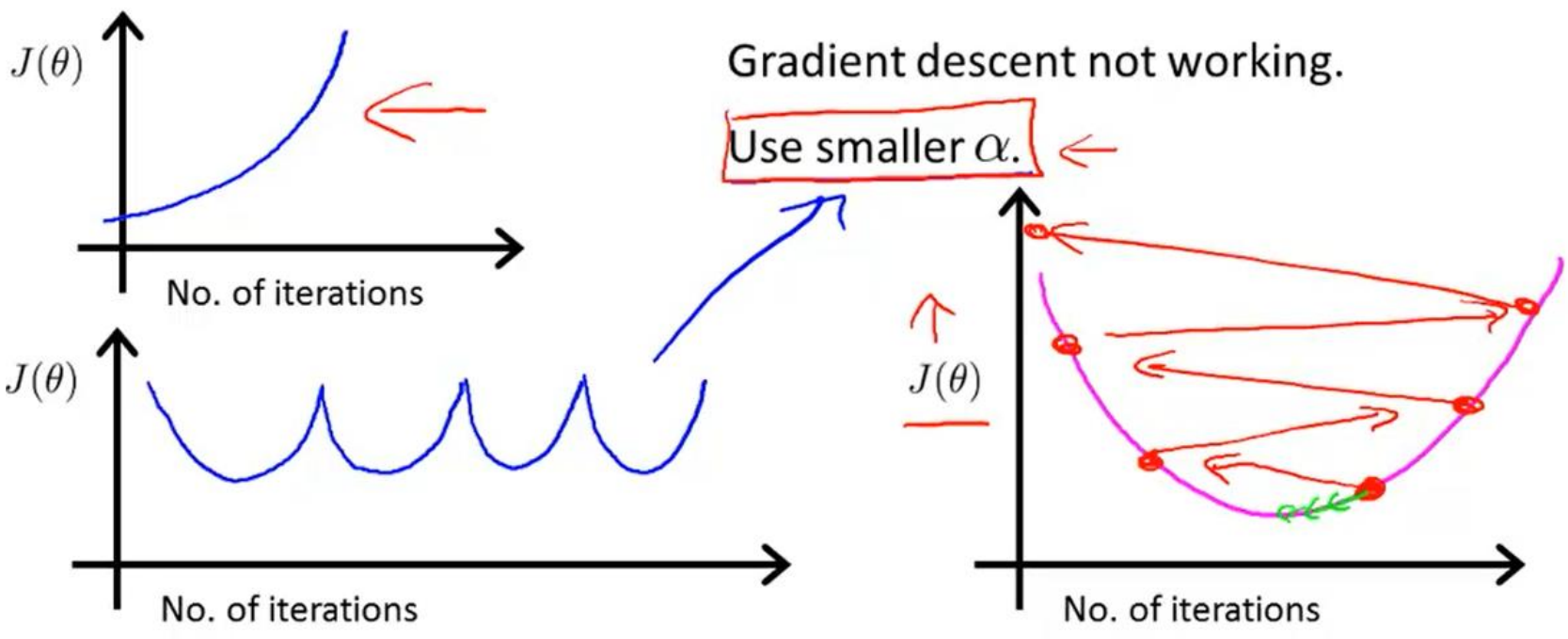
Practical Learning Rate



Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

Practical Learning Rate

- If we plot the J(θ) and see the figures like this we are supposed to decrease the learning rate.



Practical Learning Rate

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

To choose α, try

$$\ldots, 0.001, \qquad , 0.01, \qquad , 0.1, \qquad , 1, \ldots$$

Polynomial Regression

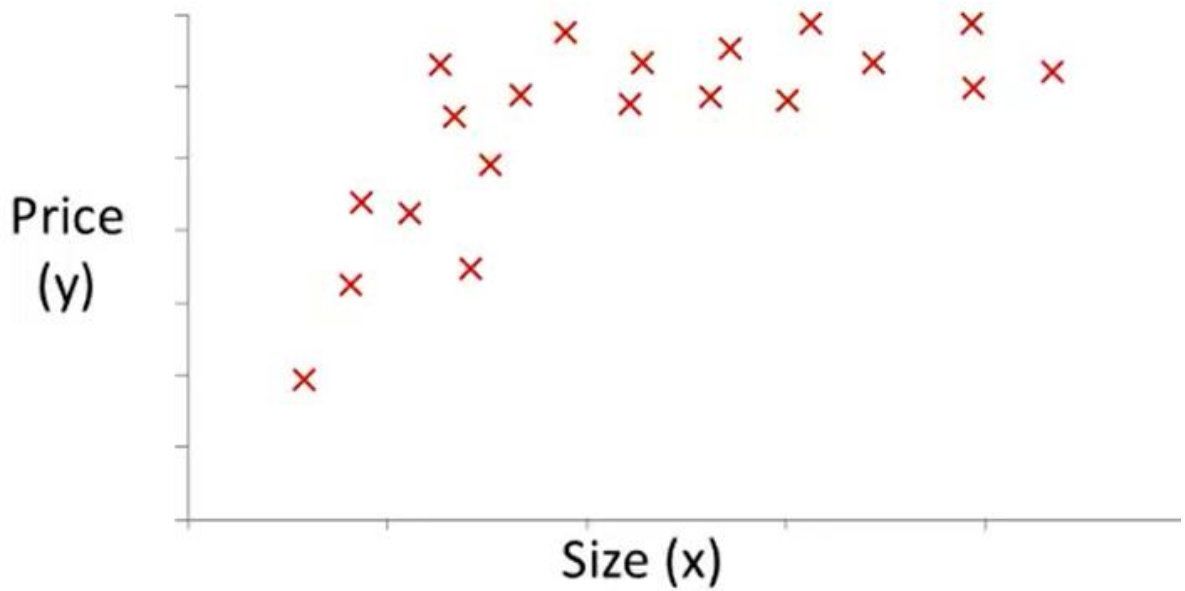
- Straight line may NOT fit the dataset appropriately
- Curve generated by quadratic function will eventually comes down (that is not suitable in our example)
- We may prefer

to use polynomial

function by

Increasing the

features



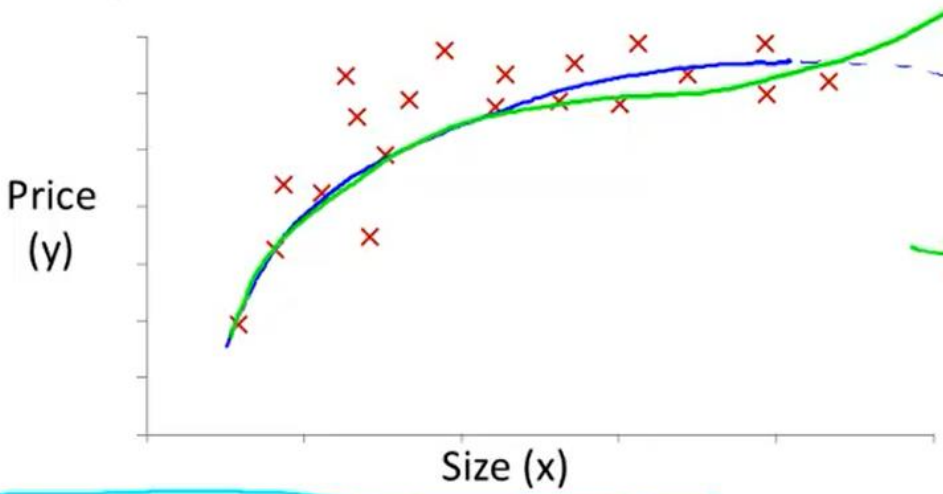
Polynomial Regression

- **Polynomial Regression** is a form of linear regression in which the relationship between the independent variable x and dependent variable y is modeled as an *nth* degree polynomial
- It is also called as multiple linear regression models in ML

Polynomial Regression

- Polynomial regression uses the relationship between the variables x and y to find the best way to draw a line through the data points
- It is a linear model with increasing accuracy
- The dataset used for multiple regression is nonlinear.

Polynomial Regression



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$= \theta_0 + \theta_1 (size) + \theta_2 (size)^2 + \theta_3 (size)^3$$

$$x_1 = (size)$$
$$x_2 = (size)^2$$
$$x_3 = (size)^3$$

Size: 1 - 1000

$Size^2$: 1 - 1000,000

$Size^3$: 1 - 10^9

Activate Windows
Go to Settings to activate Windows.

Andrew

Introducing / Creating Additional Features

Housing prices prediction

$$h_\theta(x) = \theta_0 + \theta_1 \times frontage + \theta_2 \times depth$$

Logistic Regression

- **Classification** is an area of supervised machine learning that tries to predict which class or category some entity belongs to, based on its features.
- **Independent variables**, also called inputs or predictors, don't depend on other features of interest (or at least you assume so for the purpose of the analysis).
- **Dependent variables**, also called outputs or responses, depend on the independent variables.

Logistic Regression

Email: Spam / Not Spam?
Online Transactions: Fraudulent (Yes / No)?
Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: "Negative Class" (e.g., benign tumor)
1: "Positive Class" (e.g., malignant tumor)

Logistic Regression

$h_\theta(x)$

Interpretation of Hypothesis Output

$h_\theta(x)$ = estimated probability that $\boxed{y = 1}$ on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$h_\theta(x) = 0.7$ $\quad y=1$

Tell patient that 70% chance of tumor being malignant

$h_\theta(x) = P(y=1|x;\theta)$

"probability that y = 1, given x, parameterized by θ"

$y = 0$ or 1

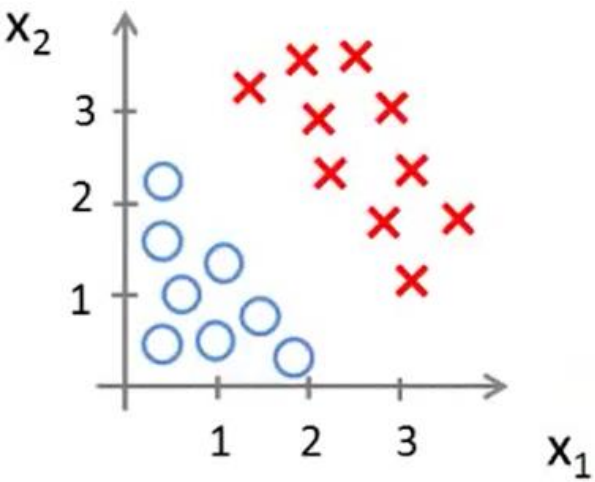
$$P(y=0|x;\theta) + P(y=1|x;\theta) = 1$$
$$P(y=0|x;\theta) = 1 - P(y=1|x;\theta)$$

Logistic Regression (Decision Boundary)

Suppose predict "$y = 1$" if $h_\theta(x) \geq 0.5$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

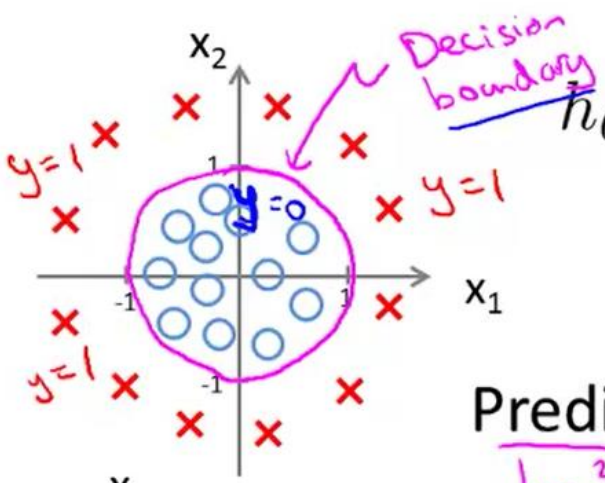
predict "$y = 0$" if $h_\theta(x) < 0.5$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Logistic Regression (Decision Boundary)

Non-linear decision boundaries

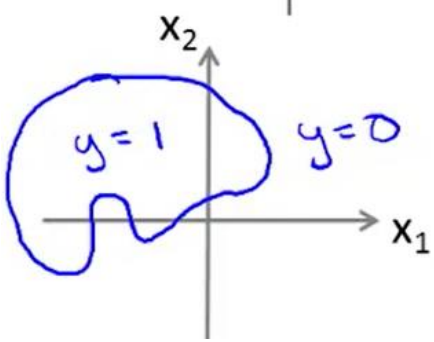


$$h_\theta(x) = g(\overset{=-1}{\theta_0} + \overset{=0}{\theta_1 x_1} + \overset{=0}{\theta_2 x_2} + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict "$y = 1$" if $\underbrace{-1 + x_1^2 + x_2^2 \geq 0}_{x_1^2 + x_2^2 \geq 1}$

$\boxed{x_1^2 + x_2^2 = 1}$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \ldots)$$

Activate Windows
Go to Settings to activate Windows.

Logistic Regression Multiclass Classification

Email foldering/tagging: Work, Friends, Family, Hobby

$y=1$ $y=2$ $y=3$ $y=4$

Medical diagrams: Not ill, Cold, Flu

$y=1$ 2 3

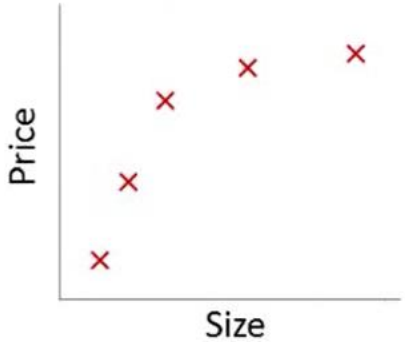
Weather: Sunny, Cloudy, Rain, Snow

$y=1$ 2 3 4

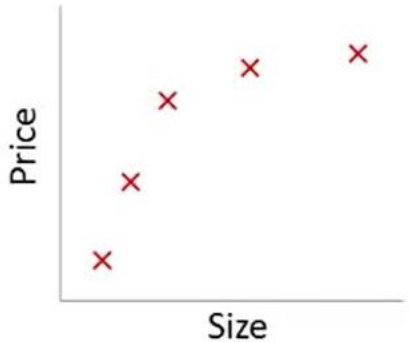
Regularization

Example: Linear regression (housing prices)

$\theta_0 + \theta_1 x$



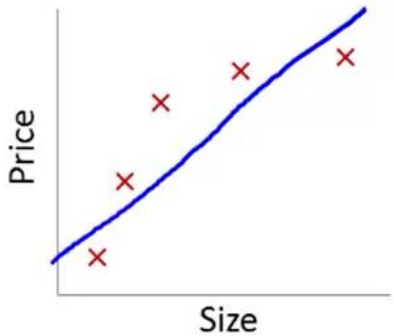
$\theta_0 + \theta_1 x + \theta_2 x^2$



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

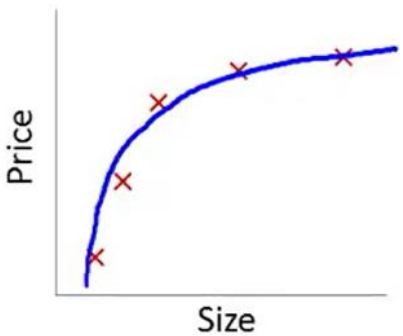
Regularization

Example: Linear regression (housing prices)

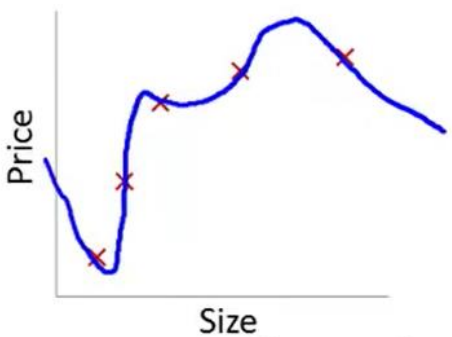


$\rightarrow \theta_0 + \theta_1 x$

"Underfit" "High bias"



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

"Overfit" "High variance"

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

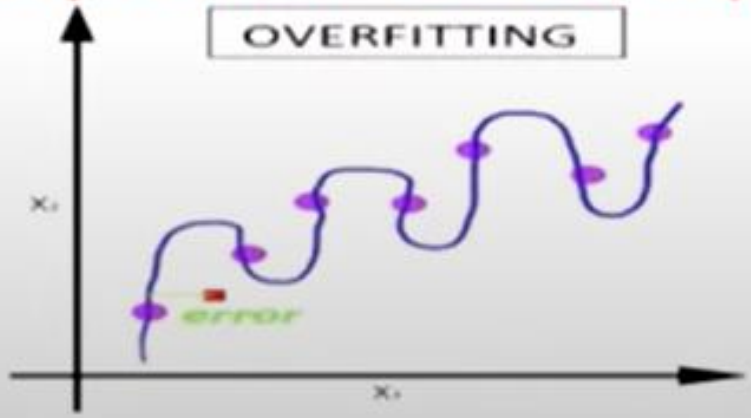
Activate Windows
Go to Settings to activate Windows.

What is Overfitting?

Too Complex: An overfit model **learns** the training data **too well**.

Memorization, not Generalization: It becomes a **lookup table** rather than a **pattern detector**.

Poor performance on new data: The model becomes overly specific to the training data and makes **bad predictions** on **unseen examples**.



Consequences of Overfitting and Underfitting

Inaccuracy: Both lead to **unreliable predictions.**

Underfitting: High bias, the model consistently gets it wrong in a certain way.

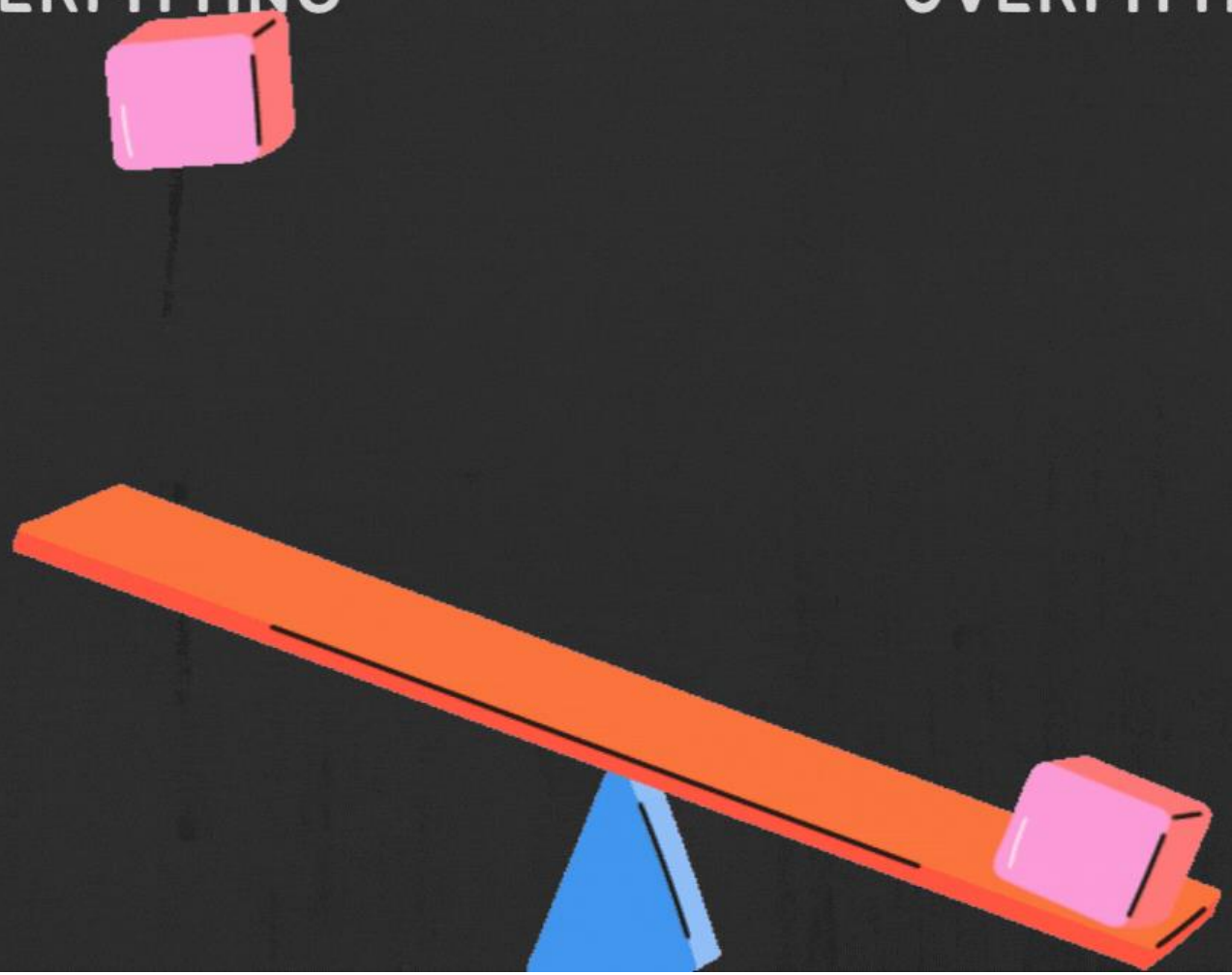
Overfitting: High variance performance wildly depends on the specific data it has seen.

High Bias

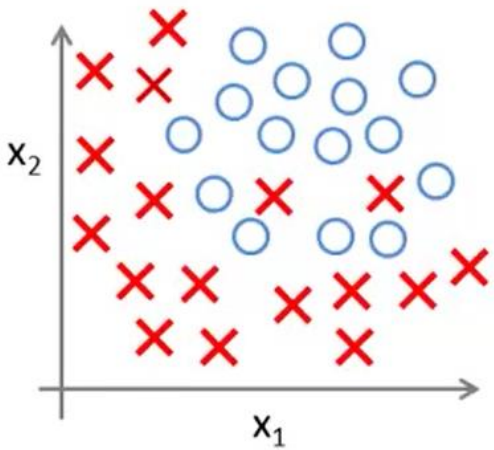
UNDERFITTING

High Variance

OVERFITTING

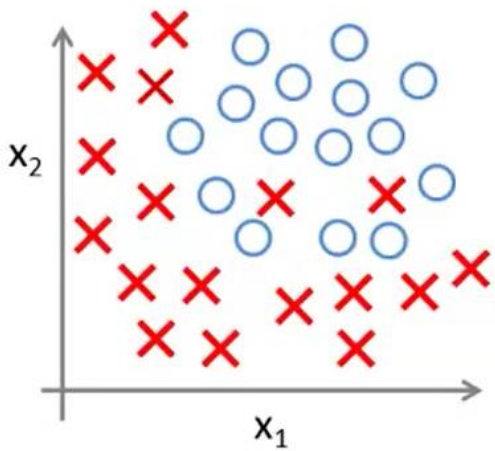


Regularization

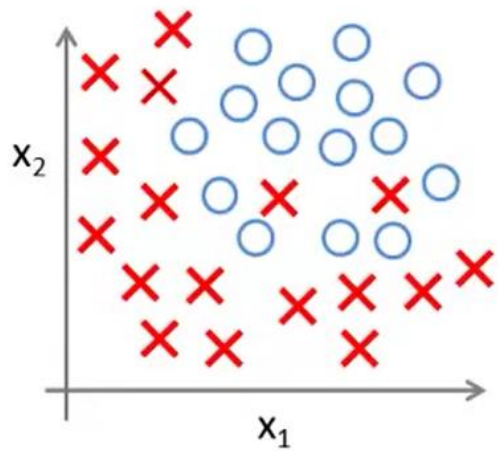


$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

(g = sigmoid function)

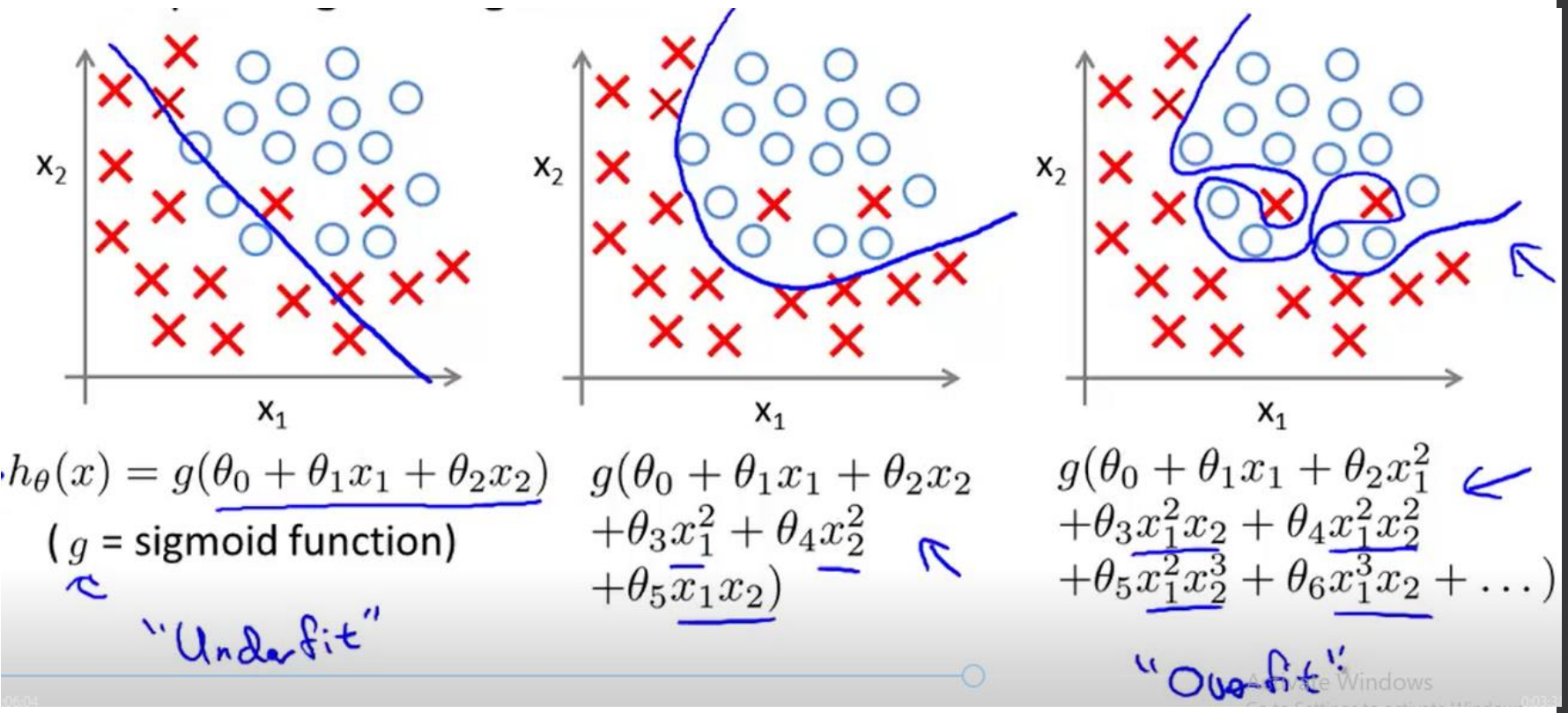


$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$

Regularization



Regularization

Addressing overfitting:

$x_1 =$ size of house
$x_2 =$ no. of bedrooms
$x_3 =$ no. of floors
$x_4 =$ age of house
$x_5 =$ average income in neighborhood
$x_6 =$ kitchen size
$\vdots$
x_{100}

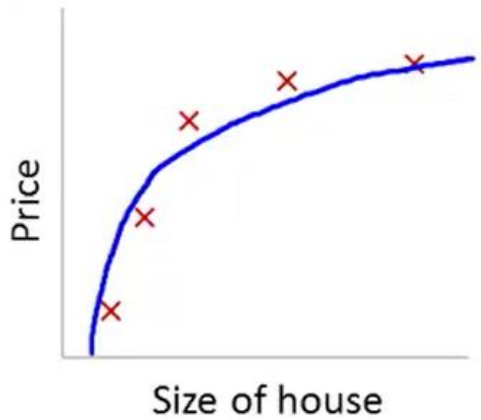
Regularization

Addressing overfitting:

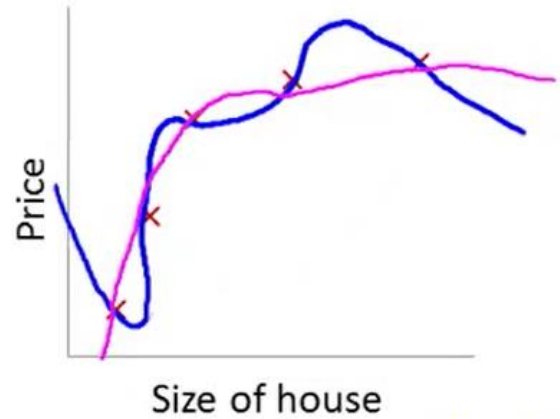
Options:

1. Reduce number of features.
   - Manually select which features to keep.
   - Model selection algorithm (later in course).
2. Regularization.
   - Keep all the features, but reduce magnitude/values of parameters θ_j.
   - Works well when we have a lot of features, each of which contributes a bit to predicting y.

Regularization



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

Suppose we penalize and make θ_3, θ_4 really small.

$$\rightarrow \min_\theta \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + 1000\,\theta_3^2 + 1000\,\theta_4^2$$

$\theta_3 \approx 0 \qquad \theta_4 \approx 0$

Activate Windows
Go to Settings to activate Wir

Ways to Tackle Overfitting

- Cross-Validation
- Training with more data
- Removing features
- Early stopping the training
- Regularization
- Ensembling

Ways to Tackle Underfitting

- Increase the number of features in the dataset
- Increase model complexity
- Reduce noise in the data
- Increase the duration of training the data

Working with data for machine learning projects

- **Handling Categorical Variables**
- Categorical variables are basically the variables that are discrete and not continuous.
- Ex — color of an item is a discrete variable whereas its price is a continuous variable.
- **Ordinal categorical variables** — These variables can be ordered. Ex — Size of a T-shirt.
- We can say that M<L<XL.
- **Nominal categorical variables** — These variables can't be ordered. Ex — Color of a T-shirt.
- We can't say that Blue<Green as it doesn't make any sense to compare the colors as they don't have any relationship

Handling Nominal Categorical Variables

- If you use the same map() function or LabelEncoder with nominal variables then the model will think that there is some sort of relationship between the nominal values
  - *col_mapping = {'Blue':1,'Green':2}*
- Then according to the model, Green > Blue, which is a senseless assumption and the model will give you results considering this relationship.

One-Hot Encoding

- So in One-Hot Encoding what we essentially do is that we create ’n’ columns where n is the number of unique values that the nominal variable can take.
- Ex — Here if color can take Blue, Green and White then we will just create three new columns namely — color_blue, color_green and color_white and if the color is green then the values of color_blue and color_white column will be 0 and value of color_green column will be 1.

One-Hot Encoding

- # example of a one hot encoding
- from numpy import asarray
- from sklearn.preprocessing import OneHotEncoder
- # define data
- data = asarray([['red'], ['green'], ['blue']])
- print(data)
- # define one hot encoding
- encoder = OneHotEncoder(sparse=False)
- # transform data
- onehot = encoder.fit_transform(data)
- print(onehot)

Artificial Data Synthesis or Data Augmentation

- Example 1: Speech recognition of car voice data.
- Example 2: Mobile phone image recognition for cat images.

Artificial Data Synthesis or Data Augmentation

- Data augmentation is the process of increasing the diverse number of samples by some computer-based techniques without actually collecting the new samples.
- For example, new image files can be created from the already available images by changing their:
  - width, height, light, brightness, zooming, cropping, padding, flipping or rotation.

Artificial Data Synthesis or Data Augmentation

- Data augmentation techniques are important to make the new images different and versatile from the original ones.
- Mostly the data augmentation is performed on images but sometimes it is also essential for increasing the data present in other formats.
- *Offline augmentation* - Used for small datasets. It is applied in the data preprocessing step.
- *Online augmentation*- Used for large datasets. It is normally applied in real-time.

Challenges of Data Synthesis

- Suppose you have 1,000 hours of speech training data, but only 1 hour of car noise.
- Alternatively, suppose you have 1,000 unique hours of car noise, but all of it was taken from just 10 different cars.

Challenges of Data Synthesis

- To take one more example, suppose you are building a computer vision system to recognize cars.
- Suppose you partner with a video gaming company, which has computer graphics models of several cars. To train your algorithm, you use the models to generate synthetic images of cars.
- What's the problem in this approach?

Challenges of Data Synthesis

- Even if the synthesized images look very realistic, this approach (which has been independently proposed by many people) will probably not work well.
- The video game might have ~20 car designs in the entire video game.

Challenges of Data Synthesis

- It is very expensive to build a 3D car model of a car; if you were playing the game, you probably wouldn't notice that you're seeing the same cars over and over, perhaps only painted differently.
- I.e., this data looks very realistic to you.

Challenges of Data Synthesis

- But compared to the set of all cars out on roads—and therefore what you're likely to see in the dev/test sets—this set of 20 synthesized cars captures only a minuscule fraction of the world's distribution of cars.

Steps in Data Preprocessing in Machine Learning

- **1. Acquire the dataset**
- To build and develop Machine Learning models, you must first acquire the relevant dataset.
- There are several online sources from where you can download datasets like https://www.kaggle.com/uciml/datasets
- **https://github.com/awesomedata/awesome-public-datasets**

Steps in Data Preprocessing in Machine Learning

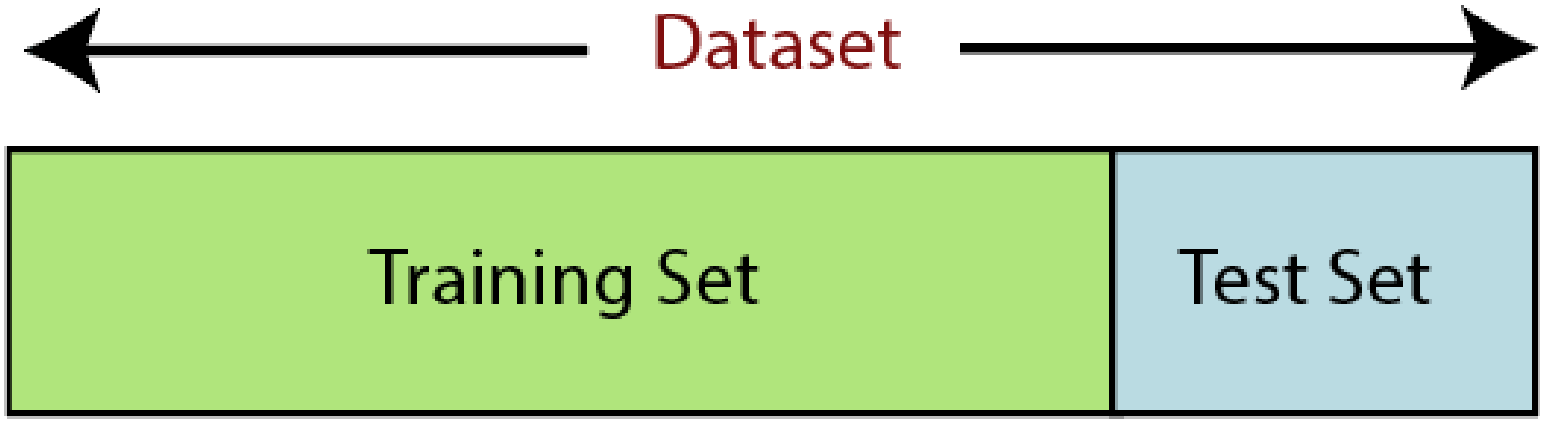
- **2. Import all the crucial libraries**
- The predefined Python libraries can perform specific data preprocessing jobs. Importing all the crucial libraries is the second step in data preprocessing in machine learning. The three core Python libraries used for this data preprocessing in Machine Learning are:
  - **NumPy** – NumPy is the fundamental package for scientific calculation in Python.
  - **Pandas** – Pandas is an excellent open-source Python library for data manipulation and analysis.
  - **Matplotlib** – Matplotlib is a Python 2D plotting library that is used to plot any type of charts in Python.

Steps in Data Preprocessing in Machine Learning

- **3. Import the dataset**
- In this step, you need to import the dataset/s that you have gathered for the ML project at hand.
- **Feature Selection**
- **4. Identifying and handling the missing values**
- **5. Encoding the categorical data**

Steps in Data Preprocessing in Machine Learning

- **6. Splitting the dataset**



Steps in Data Preprocessing in Machine Learning

- **To split the dataset, you have to write the following line of code –**
  - *from sklearn.model_selection import train_test_split*
  - *x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.2, random_state=42)*
- Here, the first line splits the arrays of the dataset into random train and test subsets.
- The second line of code includes four variables:
- x_train – features for the training data
- x_test – features for the test data
- y_train – dependent variables for training data
- y_test – independent variable for testing data

Steps in Data Preprocessing in Machine Learning

- If the 70-30 split seems odd to you, think about it like this.
- Let's say your teacher gives you a practice exam and tells you the real exam will be no different…

Steps in Data Preprocessing in Machine Learning

- We usually define:
- **Training set** — Which you run your learning algorithm on.
- **Dev (development) set** — Which you use to tune parameters, select features, and make other decisions regarding the learning algorithm. Sometimes also called the **hold-out cross validation set** .
- **Test set** — which you use to evaluate the performance of the algorithm, but not to make any decisions regarding what learning algorithm or parameters to use.

Steps in Data Preprocessing in Machine Learning

- Your test set should not simply be 30% of the available data.
- For example cat app.

Steps in Data Preprocessing in Machine Learning

- An analogy:
- You have your cat app image data segmented into four regions, based on your largest markets: (i) US, (ii) China, (iii) India, and (iv) Other.
- To come up with a dev set and a test set, say we put US and India in the dev set; China and Other in the test set.

Steps in Data Preprocessing in Machine Learning

- Suppose your team develops a system that works well on the development set but not the test set (the dev and test sets come from same distributions).
- Where's the problem?
- You have overfit the dev set.
- The obvious cure is to get more dev set data.
- And what if the dev and test sets come from different distributions?

Steps in Data Preprocessing in Machine Learning

- 1. You had overfit to the dev set.
- 2. The test set is harder than the dev set. So your algorithm might be doing as well as could be expected, and no further significant improvement is possible.
- 3. The test set is not necessarily harder, but just different, from the dev set. So what works well on the dev set just does not work well on the test set.

Steps in Data Preprocessing in Machine Learning

- **7. Feature scaling**
- Feature scaling marks the end of the data preprocessing in Machine Learning.
- It is a method to standardize the independent variables of a dataset within a specific range.

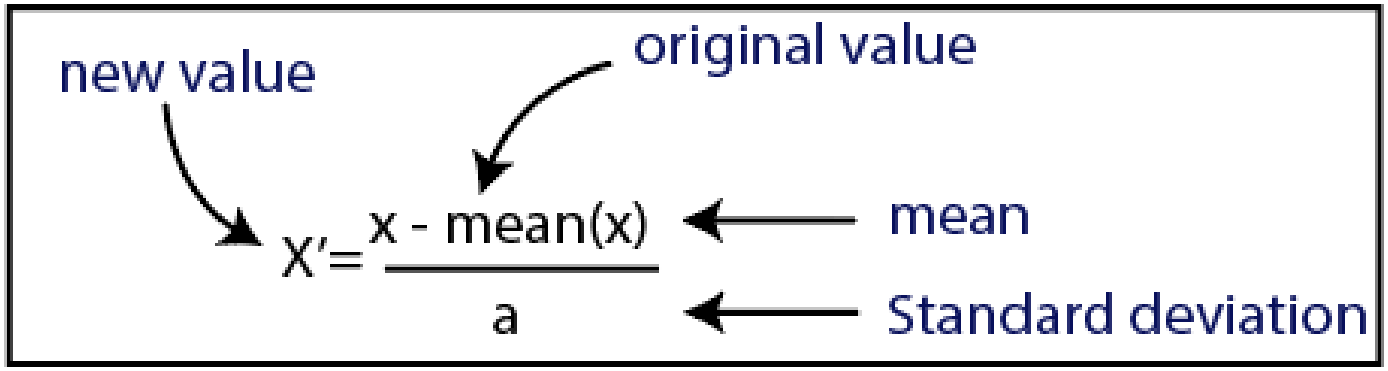
Steps in Data Preprocessing in Machine Learning

data_set - DataFrame

Index	Country	Age	Salary	Purchased
0	India	38	68000	No
1	France	43	45000	Yes
2	Germany	30	54000	No
3	France	48	65000	No
4	Germany	40	nan	Yes
5	India	35	58000	Yes
6	Germany	nan	53000	No
7	France	49	79000	Yes
8	India	50	88000	No
9	France	37	77000	Yes

Steps in Data Preprocessing in Machine Learning

- You can perform feature scaling in Machine Learning in two ways:
- **Standardization**



Steps in Data Preprocessing in Machine Learning

- **Normalization**

new value

original value

$$X' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Steps in Data Preprocessing in Machine Learning (Optional)

- *from sklearn.preprocessing import StandardScaler*
- *st_x= StandardScaler()*
- *x_train= st_x.fit_transform(x_train)*
- *x_test= st_x.transform(x_test)*

Steps in Data Preprocessing in M

x_train - NumPy array

	0	1	2	3	4
0	-1	1.73205	-0.57735	-0.294607	0.133962
1	1	-0.57735	-0.57735	-0.930959	1.22627
2	1	-0.57735	-0.57735	0.341745	-1.7415
3	-1	1.73205	-0.57735	-0.0589215	-0.999562
4	1	-0.57735	-0.57735	1.61445	1.41175
5	1	-0.57735	-0.57735	1.40233	0.113352
6	-1	-0.57735	1.73205	-0.718842	0.391581
7	-1	-0.57735	1.73205	-1.35519	-0.535848

Format Resize ☑ Background color

Basics - Training, Validation, and Testing Data

Training Data: We feed this data into a machine learning algorithm to develop a model.

Validation Data: This dataset tunes model parameters (hyperparameters), helping prevent overfitting.

Testing Data: Provides a final, unbiased picture of how well the model performs on unseen data.